

CTI ADP Software: Test Environment Specification

Version: 1.0 | **Date:** 2026-05-04 | **Status:** Active

1. OVERVIEW

This document specifies the test environments required for all projects in the CTI ADP Software suite. It covers hardware requirements, software dependencies, database setup, and configuration for each project.

Projects Covered

- 1. **E-Commerce Frontend** (React/TypeScript/Vite)
- 2. **E-Commerce Backend** (Express.js/TypeScript)
- 3. **Mobile App** (Android/Kotlin)
- 4. **Desktop App** (JavaFX/Java)

2. ENVIRONMENT TYPES

Environment	Purpose	Access	Data
Local/Development	Developer machine testing	Private	Test fixtures
CI/CD Pipeline	Automated testing on PR	GitHub Actions	Test database
Staging	Pre-production validation	Internal	Staging data
Production	Live system	Public	Real user data

Focus of this doc: Local & CI/CD environments

3. HARDWARE REQUIREMENTS

3.1 Minimum Requirements (All Projects)

Component	Minimum	Recommended
CPU	4 cores	8+ cores
RAM	8 GB	16+ GB
Disk	50 GB SSD	100+ GB SSD
Network	10 Mbps	50+ Mbps

3.2 Per-Project

Project	CPU	RAM	Disk	Notes
Frontend	2 cores	2 GB	5 GB	Lightweight, Vite fast
Backend	2 cores	2 GB	10 GB	Node + database connections
Mobile	4 cores	4 GB	20 GB	Android SDK, emulator
Desktop	2 cores	2 GB	10 GB	JavaFX rendering

4. SOFTWARE REQUIREMENTS

4.1 All Projects

Software	Version	Purpose
Git	2.40+	Version control
Node.js	18.x (LTS)	Frontend/Backend runtime
npm	9.x+	Package manager

4.2 E-Commerce Frontend

Software	Version	Purpose
Node.js	18.x	Runtime
npm	9.x	Package manager
TypeScript	5.3+	Type checking
Vite	5.0+	Build tool
Vitest	4.1+	Test runner

Installation:

```
# macOS (brew)
brew install node@18
```

```
# Windows (choco)
choco install nodejs
```

```
# Verify
node --version    # v18.x.x
npm --version     # 9.x.x
```

4.3 E-Commerce Backend

Software	Version	Purpose
Node.js	18.x	Runtime
npm	9.x	Package manager
PostgreSQL	15+	Database
TypeScript	5.3+	Type checking
Jest	29.7+	Test runner

Installation:

```
# Backend node dependencies already in package.json
# PostgreSQL setup via Docker (see Section 6)
```

4.4 Mobile App (Android)

Software	Version	Purpose
Java	17+	Runtime
Android SDK	34 (API level)	Build tools
Gradle	8.2+	Build system
Kotlin	1.9.0+	Language
Android Studio	2023.1+	IDE (optional)

Installation:

```
# Windows: Download Android Studio
# https://developer.android.com/studio

# Set ANDROID_HOME
# Windows: $env:ANDROID_HOME = "C:\Users\USERNAME\AppData\Local\Android\sdk"

# Verify
gradle --version
```

4.5 Desktop App (JavaFX)

Software	Version	Purpose
Java	17+	Runtime
Maven	3.8+	Build system
PostgreSQL	15+	Database
JavaFX	21	UI framework

Installation:

```
# Windows (choco)
choco install openjdk maven

# Verify
java --version
mvn --version
```

5. DATABASE SETUP

5.1 PostgreSQL (Docker - Recommended)

Start Database with Docker Compose

```
cd e-commerce
docker-compose up -d db
```

What it does:

- Pulls postgres:15-alpine image
- Creates database: cti_ecommerce
- User: cti_user / Password: cti_password_dev
- Port: 5432 (container) → 5433 (host)
- Initializes schema from assets/db/lpa_ecomms_schema.sql
- Seeds test data from assets/db/mock_data.sql

Verify Database Connection

```
# From host machine
psql -h localhost -p 5433 -U cti_user -d cti_ecommerce -c "SELECT 1"

# Should output: 1 (indicating successful connection)
```

Database Credentials

Property	Value
Host	localhost

Property	Value
Port	5433
Database	cti_ecommerce
Username	cti_user
Password	cti_password_dev

View Database Data

```
# Connect to database
docker exec -it cti-postgres-db psql -U cti_user -d cti_ecommerce

# Common queries
\dt          # List tables
SELECT * FROM products;
SELECT * FROM users;
\q          # Quit
```

5.2 PostgreSQL (Local Installation - Alternative)

Windows Installation

```
# Download PostgreSQL 15
# https://www.postgresql.org/download/windows/

# Or use choco
choco install postgresql

# During installation:
# - Port: 5432
# - User: postgres
# - Password: (set secure password)
```

Create Test Database

```
# Connect as admin
psql -U postgres

# Commands:
CREATE USER cti_user WITH PASSWORD 'cti_password_dev';
CREATE DATABASE cti_ecommerce OWNER cti_user;
GRANT ALL PRIVILEGES ON DATABASE cti_ecommerce TO cti_user;

# Exit
\q
```

Initialize Schema

```
# Run schema script
psql -U cti_user -d cti_ecommerce -f assets/db/lpa_ecomms_schema.sql

# Load test data
psql -U cti_user -d cti_ecommerce -f assets/db/mock_data.sql
```

6. PROJECT SETUP & CONFIGURATION

6.1 E-Commerce Frontend Setup

Installation

```
cd e-commerce/frontend
npm install
```

Environment Variables

Create .env.test file:

```
VITE_API_BASE_URL=http://localhost:3000/api
VITE_ENV=test
```

Configuration Files

- vite.config.ts — Build configuration
- vitest.config.ts — Test runner configuration (jsdom environment)
- tsconfig.json — TypeScript configuration

Start Development Server

```
npm run dev
# Runs on http://localhost:5173
```

Run Tests

```
npm test                # Watch mode
npm run test:coverage    # With coverage report
npm run test:ui          # Interactive UI
```

6.2 E-Commerce Backend Setup

Installation

```
cd e-commerce/backend
npm install
npm run prisma:generate    # Generate Prisma client
```

Environment Variables

Create .env file (already exists):

```
NODE_ENV=development
BACKEND_PORT=3000
DATABASE_URL=postgresql://cti_user:cti_password_dev@localhost:5433/cti_ecommerce?schema=public
JWT_SECRET=my_super_secret_key_here_make_it_long_and_secure
```

Important:

- DATABASE_URL uses port 5433 (Docker container mapping)
- If using local PostgreSQL, change to 5432

- Use strong JWT_SECRET in production

Configuration Files

- jest.config.js — Test runner (Node environment)
- tsconfig.json — TypeScript configuration
- prisma/schema.prisma — Database schema

Start Development Server

```
npm run dev
# Runs on http://localhost:3000
# Auto-restarts on file changes
```

Run Tests

```
npm test                # Watch mode
npm run test:watch       # Same as above
npm test -- --coverage   # With coverage report
```

Database Migrations

```
npm run prisma:migrate   # Apply pending migrations
npm run prisma:studio    # Visual database editor
```

6.3 Mobile App Setup

Installation

```
cd mobile
```

Android SDK Setup

```
# Set Android SDK path
$env:ANDROID_HOME = "C:\Users\%env:USERNAME%\AppData\Local\Android\sdk"

# Download SDK components (via Android Studio)
# - Android 14 (API 34)
# - Build Tools 34.0.0
# - Android Emulator (optional)
```

Configuration Files

- build.gradle (project root) — Root configuration
- app/build.gradle — App-specific configuration
- gradle.properties — Gradle properties

Build & Run

```
# Build debug APK
./gradlew build

# Run tests
./gradlew test                # Unit tests (JUnit4)
./gradlew connectedAndroidTest # UI tests (Espresso)

# Run on emulator/device
./gradlew installDebug
```

Android Emulator (Optional)

```
# Create emulator
android create avd -n test-emulator -t android-34

# Start emulator
$ANDROID_HOME/emulator/emulator -avd test-emulator

# Or use Android Studio GUI
```

6.4 Desktop App Setup

Installation

```
cd desktop
```

Java & Maven Setup

```
# Verify Java 17+
```

```
java --version
```

```
# Verify Maven 3.8+
```

```
mvn --version
```

```
# If needed, set JAVA_HOME
```

```
$env:JAVA_HOME = "C:\Program Files\Java\jdk-17"
```

Configuration Files

- pom.xml — Maven project configuration
- src/main/resources/ — Application resources

Build & Run

```
# Build project
```

```
mvn clean compile
```

```
# Run tests
```

```
mvn test
```

```
# Build executable JAR
```

```
mvn clean package
```

```
# Run application
```

```
java -jar target/stock-management-1.0-SNAPSHOT.jar
```

Coverage Report

```
# Generate JaCoCo coverage report  
mvn jacoco:report
```

```
# View report  
# target/site/jacoco/index.html
```

7. CI/CD PIPELINE SETUP

7.1 GitHub Actions Workflow

File: .github/workflows/test.yml

name: Tests

on: [pull_request, push]

jobs:

test:

runs-on: ubuntu-latest

services:

postgres:

image: postgres:15-alpine

env:

POSTGRES_USER: cti_user

POSTGRES_PASSWORD: cti_password_dev

POSTGRES_DB: cti_ecommerce

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

ports:

- 5432:5432

steps:

- uses: actions/checkout@v3

- name: Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '18'

- name: Setup Java

uses: actions/setup-java@v3

with:

distribution: 'temurin'

java-version: '17'

Frontend Tests

- name: Frontend - Install Dependencies

run: cd e-commerce/frontend && npm install

- name: Frontend - Type Check

run: cd e-commerce/frontend && npm run type-check

```
- name: Frontend - Lint
  run: cd e-commerce/frontend && npm run lint

- name: Frontend - Test
  run: cd e-commerce/frontend && npm test -- --run --coverage

# Backend Tests
- name: Backend - Install Dependencies
  run: cd e-commerce/backend && npm install

- name: Backend - Generate Prisma Client
  run: cd e-commerce/backend && npm run prisma:generate

- name: Backend - Type Check
  run: cd e-commerce/backend && npm run type-check

- name: Backend - Lint
  run: cd e-commerce/backend && npm run lint

- name: Backend - Test
  run: cd e-commerce/backend && npm test -- --coverage
  env:
    DATABASE_URL: postgresql://cti_user:cti_password_dev@localhost:5432/cti_ecommerce?schema=public

# Mobile Tests
- name: Mobile - Build
  run: cd mobile && ./gradlew build

- name: Mobile - Test
  run: cd mobile && ./gradlew test

# Desktop Tests
- name: Desktop - Build
  run: cd desktop && mvn clean compile

- name: Desktop - Test
  run: cd desktop && mvn test

# Upload Coverage
- name: Upload Coverage
  uses: codecov/codecov-action@v3
```

```
with:
  files: ./coverage/coverage-final.json
```

7.2 Pre-Commit Hooks (Optional)

File: .husky/pre-commit

```
#!/bin/sh
. "$(dirname "$0")/_/husky.sh"

npm run type-check
npm run lint -- --fix
npm test -- --onlyChanged
```

8. NETWORK & CONNECTIVITY

8.1 Local Development Network

Developer Machine (localhost)	
Port 3000:	Backend API
Port 5173:	Frontend (Vite)
Port 5432:	PostgreSQL (Docker)
Port 5433:	PostgreSQL (Container)
Port 6379:	Redis (optional)

8.2 Connection Validation

Frontend → Backend API

```
# Test from frontend directory
curl http://localhost:3000/api/health

# Expected response: { "status": "ok" }
```

Backend → Database

```
# Test from backend directory
npm run prisma:studio

# Or run a test that queries database
npm test -- --testNamePattern="database"
```

Mobile → Backend (WebView)

WebView loads: http://localhost:3000
(When running on physical device or emulator with proper network config)

9. SECURITY CONFIGURATION

9.1 Environment Variables Management

Never commit: .env , .env.local , .env.*.local

Secure files:

```
# .env.example (safe to commit - template only)
NODE_ENV=development
BACKEND_PORT=3000
DATABASE_URL=postgresql://user:password@host:port/db
JWT_SECRET=change_me_in_development
```

9.2 Local Development Security

Item	Recommendation
Database Password	Use weak password (dev only)
JWT Secret	Use a long random string (≥32 chars)
API Keys	Use test/staging keys
Network	Only localhost access in dev

Generate Strong Secrets:

```
# Frontend/Backend
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# Output example:
# a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6
```

10. TEST DATA & FIXTURES

10.1 Test Database Setup

Automatically initialized via Docker:

```
docker-compose.yml
├─ Schema: assets/db/lpa_ecomms_schema.sql
└─ Data: assets/db/mock_data.sql
```

10.2 Test Data Cleanup

Between Test Runs (Recommended)

```
-- Reset auto-increment sequences
ALTER SEQUENCE products_id_seq RESTART WITH 1;
ALTER SEQUENCE users_id_seq RESTART WITH 1;
ALTER SEQUENCE orders_id_seq RESTART WITH 1;

-- Clear data (if desired)
DELETE FROM orders;
DELETE FROM cart_items;
DELETE FROM products;
DELETE FROM users;
```

Per-Test Cleanup (Better Practice)

Each test creates isolated data and cleans up after:

```
// Jest test example
afterEach(() => {
  // Clean up test data
  prisma.product.deleteMany({
    where: { sku: { startsWith: "TEST_" } }
  });
});
```

10.3 Test Fixtures Location

```
e-commerce/backend/src/__tests__/fixtures/
├─ users.fixture.ts
├─ products.fixture.ts
├─ orders.fixture.ts
└─ ...
```

```
e-commerce/frontend/src/__tests__/fixtures/
├─ cart.fixture.ts
├─ product.fixture.ts
└─ ...
```

11. TROUBLESHOOTING

11.1 Common Issues

Issue: Database Connection Refused

Symptoms: Error: connect ECONNREFUSED 127.0.0.1:5433

Solution:

```
# Check if Docker container running
docker ps | grep cti-postgres-db

# If not running, start it
docker-compose up -d db

# Check logs
docker logs cti-postgres-db
```

Issue: Port Already in Use

Symptoms: Error: listen EADDRINUSE :::3000

Solution:

```
# Find process using port 3000
lsof -i :3000 # macOS/Linux
netstat -ano | findstr :3000 # Windows

# Kill process
kill -9 <PID> # macOS/Linux
taskkill /PID <PID> /F # Windows

# Or use different port
BACKEND_PORT=3001 npm run dev
```

Issue: TypeScript Compilation Error

Symptoms: error TS2307: Cannot find module '@application/...'

Solution:

```
# Regenerate TypeScript
tsc --noEmit

# Or in tsconfig.json, verify paths:
"baseUrl": ".",
"paths": {
  "@application/*": ["src/application/*"],
  "@domain/*": ["src/domain/*"]
}
```

Issue: Tests Timing Out

Symptoms: Jest: Timeout - Async callback was not invoked

Solution:

```
// Increase timeout
jest.setTimeout(10000); // 10 seconds

// Or use proper async/await
test('async test', async () => {
  const result = await someAsyncFunction();
  expect(result).toBeDefined();
});
```

Issue: Mobile Emulator Not Starting

Symptoms: Emulator failed to start

Solution:

```
# Delete corrupted emulator
$ANDROID_HOME/emulator/emulator -avd test-emulator -wipe-data

# Recreate
android create avd -n test-emulator -t android-34

# Restart with more RAM
emulator -avd test-emulator -memory 2048
```

11.2 Health Check Commands

Frontend

```
npm run type-check    # TypeScript
npm run lint          # ESLint
npm test -- --run     # Single test run
```

Backend

```
npm run type-check    # TypeScript
npm run lint          # ESLint
npm test -- --coverage --testTimeout=5000
npm run prisma:studio # DB visual check
```

Mobile

```
./gradlew clean build    # Full rebuild
./gradlew test           # Unit tests
./gradlew connectedAndroidTest # UI tests (device/emulator needed)
```

Desktop

```
mvn clean compile
mvn test
mvn clean package
```

12. ENVIRONMENT CHECKLIST

Pre-Testing Checklist

- ☐ **Git** — Repository cloned, `main` branch up-to-date
- ☐ **Node.js** — Version 18.x installed (`node --version`)
- ☐ **npm** — Version 9.x installed (`npm --version`)
- ☐ **Java** — Version 17+ installed (`java --version`)
- ☐ **Maven** — Version 3.8+ installed (`mvn --version`)
- ☐ **Docker** — Running (`docker ps`)
- ☐ **PostgreSQL** — Container up (`docker-compose ps`)
- ☐ **Environment Files** — `.env` files configured
- ☐ **Dependencies** — All `npm install` completed per project
- ☐ **Database** — Schema initialized, test data loaded
- ☐ **Network** — Ports available (3000, 5173, 5432, 5433)
- ☐ **Git Hooks** — Pre-commit hooks configured (optional)

Test Execution Checklist

- ☐ Frontend: `npm test` passing in `e-commerce/frontend/`
- ☐ Backend: `npm test` passing in `e-commerce/backend/`
- ☐ Mobile: `./gradlew test` passing in `mobile/`
- ☐ Desktop: `mvn test` passing in `desktop/`
- ☐ Coverage reports generated
- ☐ No type errors across all projects
- ☐ Lint checks pass

13. QUICK START GUIDE

First-Time Setup (≈15 minutes)

1. Clone and navigate

```
git clone <repo>
cd cti-adp-software
```

2. Start database

```
cd e-commerce
docker-compose up -d db
cd ..
```

3. Setup frontend

```
cd e-commerce/frontend
npm install
npm test # Should pass
```

4. Setup backend

```
cd ../backend
npm install
npm run prisma:generate
npm test # Should pass
```

5. Setup mobile

```
cd ../../mobile
./gradlew build
./gradlew test # Should pass
```

6. Setup desktop

```
cd ../desktop
mvn clean compile
mvn test # Should pass
```

```
echo "✅ All environments ready!"
```

Daily Workflow

```
# Before starting work
git pull origin main
docker-compose up -d db # If not running

# Run tests in project directory
npm test # (Frontend/Backend)
./gradlew test # (Mobile)
mvn test # (Desktop)

# Commit only after all pass
git add .
git commit -m "feat: description"
git push origin feature-branch
```

14. REFERENCE

Port Assignments

Service	Port	Protocol
Frontend	5173	HTTP
Backend API	3000	HTTP
PostgreSQL (host)	5433	TCP
PostgreSQL (container)	5432	TCP
Redis (optional)	6379	TCP

Key Directories

```
cti-adp-software/  
├─ e-commerce/  
│   ├─ frontend/      # React app  
│   ├─ backend/       # Express.js API  
│   └─ docker-compose.yml # Database container  
├─ mobile/            # Android app  
├─ desktop/           # JavaFX app  
└─ assets/db/         # Database schemas & fixtures
```

Documentation Files

- TEST_STRATEGY.md — Testing standards and tools
- TEST_PLAN.md — Detailed test cases
- TEST_ENVIRONMENT.md — This document

Document Owner: DevOps/QA Lead

Last Updated: 2026-05-04

Next Review: 2026-06-04